By now we've seen all the data types (and pseudo-types) supported by PHP. Notice anything conspicuously absent? There is no native data type (pseudo or otherwise) for representing time (unlike, say, MySQL). Instead, we have an entire core extension (the `date` module, which will be installed by default) devoted to managing dates and times. This module provides a number of functions and an OOP API, and it's the de facto standard way of handling time in PHP code.

The Unix Epoch

Measuring time is one problem: knowing how much time has elapsed since you last looked is simple if you have a clock, which all computer systems do. Knowing the date and time, knowing *when* you are, is another problem, and a critical one. As systems became more integrated and distributed, the need to synchronize events, keep accurate logs, and so on grew. It's the train timetable problem, but this time with silicon. To solve the problem, an *epoch* date and time is chosen, and time is measured as the integer number of seconds elapsing since that epoch. There are a number of computing epochs (Windows and the `C#` programming language use 1 *CE* or 1601 *CE* depending on version), but the most widely used is the Unix Epoch, which is UTC midnight, January 1, 1970 – or `01-JAN-1970 00:00:00` or `1970-01-01 00:00:00`, or… ah, look at that. There are so many ways to express a date and time. Who knew?! In fact, why don't we have a little preview of what's to come? We'll use the `date` module's OOP API, which provides the core `DateTime` class:

```
$epoch = new DateTime('1970-01-01 00:00:00');
var_dump($epoch->format(DateTime::ATOM));
var_dump($epoch->format(DateTime::COOKIE));
var_dump($epoch->format(DateTime::RFC7231));
var_dump($epoch->format(DateTime::W3C));
```

```
Output:
string(25) "1970-01-01T00:00:00+00:00"
string(34) "Thursday, 01-Jan-1970 00:00:00 UTC"
string(29) "Thu, 01 Jan 1970 00:00:00 GMT"
string(25) "1970-01-01T00:00:00+00:00"
```